

Session 1 – Introduction to Web Applications

- Applications are programs designed for specific tasks and are classified into desktop, network, and Web applications.
- Desktop applications run locally on a single computer, while Web applications run on remote servers through browsers.
- Web applications allow users to access services globally using an Internet connection and a Web browser.
- Static Web pages display fixed content, while dynamic Web pages generate changing content based on user interaction.
- Major advantages of Web applications include lower maintenance cost, easier deployment, platform independence, and worldwide accessibility.
- Single-Page Applications (SPA) dynamically update content without reloading the entire page, improving speed and user experience.
- Microservices architecture divides applications into smaller independent services that are easier to manage and scale.
- Progressive Web Applications (PWA) behave like mobile apps and support offline access, reliability, and faster performance.
- Frontend components handle user interaction using technologies such as HTML, CSS, and JavaScript.
- Backend components process business logic, database operations, authentication, and server-side application functionality.
- HTTP is a stateless protocol used for communication between clients and Web servers through requests and responses.
- HTTP requests contain request lines, methods, URLs, headers, and sometimes body data sent to the server.
- HTTP responses contain status codes, response headers, and data returned from the server to the browser.
- GET retrieves data from servers, while POST sends large or sensitive data securely in the request body.
- PUT uploads or updates resources, DELETE removes resources, and PATCH modifies specific parts of resources.
- HTTP/3 improves performance through reduced latency, stronger security, and faster communication using the QUIC protocol.
- Servlets are Java classes used to process client requests and generate dynamic Web application responses.
- JSP allows embedding Java code into HTML pages to simplify dynamic Web page development.
- Web containers manage servlet execution, request handling, security, deployment, and communication between components.
- Servlet lifecycle includes initialization using `init()`, request processing using `service()`, and cleanup using `destroy()` methods.

Session 2 – Introduction to Jakarta EE

- Jakarta EE is an open-source enterprise Java platform previously known as Java EE before moving to Eclipse Foundation.
- Jakarta EE simplifies enterprise Web application development using reusable APIs, specifications, and container-managed services.
- Enterprise applications developed using Jakarta EE focus on scalability, security, reliability, and distributed application support.
- Jakarta EE supports annotation-based configuration, reducing XML configuration complexity and simplifying application development.

- Contexts and Dependency Injection (CDI) automatically injects dependencies into objects, creating loosely coupled applications.
- Dependency Injection improves code readability, testing, maintainability, reusability, and flexibility during application development.
- Jakarta EE 10 supports cloud-native application development and is compatible with Java SE 11, 17, and 21.
- Jakarta EE architecture uses three major tiers: presentation tier, business tier, and data persistence tier.
- The presentation tier manages user interfaces, while the business tier contains application logic and processing functionality.
- The data tier manages databases, storage systems, and enterprise information systems used by applications.
- Application Client Container connects client-side applications to Jakarta EE server-side components and enterprise services.
- Web Container manages servlets, JSP pages, and Web components responsible for processing user requests and responses.
- Business Container manages Enterprise Java Beans (EJB) and CDI components responsible for enterprise business logic.
- Jakarta EE servers provide deployment, transaction management, security, persistence, naming, and messaging services.
- Jakarta EE profiles include Core Profile, Web Profile, and Full Platform profile for different development requirements.
- Core Profile supports lightweight applications, while Web Profile supports servlet and JSP-based Web development.
- Full Platform profile contains all Jakarta EE specifications and enterprise-level services required for large applications.
- Jakarta EE applications are built using reusable components that communicate through well-defined APIs and interfaces.
- Jakarta EE supports microservices architecture using modular design, lightweight runtimes, REST APIs, and containerization support.
- Jakarta EE integrates well with Docker, Kubernetes, DevOps practices, Continuous Integration, and Continuous Deployment pipelines.
- Major advantages of Jakarta EE include platform independence, strong security, scalability, cloud-native support, and standardized APIs.
- Major disadvantages include higher infrastructure cost, steep learning curve, migration complexity, and heavyweight implementation for smaller projects.
- Important Jakarta EE specifications include Servlets, JSP, CDI, Persistence API, EJB, WebSocket, Messaging, and RESTful Web Services.
- The JDK is required for Jakarta EE development because it provides the JVM, compiler, runtime environment, and development tools.
- JAVA_HOME environment variable is configured to point to the installed JDK directory for proper Java setup.
- Eclipse IDE is a popular development environment supporting Java, Python, C++, PHP, and enterprise Web development.
- Eclipse IDE features include code completion, debugging tools, plug-ins, project management, and multi-platform support.

Session 3 – Developing and Deploying a Web Application

- Apache Tomcat is an open-source Web server and servlet container used for running JSP and servlet applications.
- Dynamic Websites require both an HTTP server and a servlet container to process requests and generate responses.
- Tomcat processes JSP pages by internally converting them into servlets before executing them on the server.

- Major advantages of Apache Tomcat include platform independence, lightweight execution, SSL support, and extensive documentation.
- Web application lifecycle includes development, deployment descriptor creation, compilation, packaging, deployment, and execution through URLs.
- A context root is the base URL used to access resources and components inside a deployed Web application.
- Web applications are packaged as WAR files to simplify deployment, portability, and transfer across servers and environments.
- The WEB-INF directory stores important deployment files such as web.xml, libraries, and servlet configuration information.
- The web.xml deployment descriptor maps URLs to servlets and stores application configuration settings using XML.
- The <servlet> element declares servlet information, while <servlet-mapping> connects servlet classes to URL patterns.
- From Java EE 6 onward, annotations reduced dependency on web.xml by embedding configuration metadata directly in source code.
- Eclipse IDE simplifies Web application development through project management, compilation, deployment, and server integration tools.
- Tomcat can be configured directly inside Eclipse for easier testing, automated deployment, and application execution management.
- Eclipse Dynamic Web Projects organize files into directories such as src, main/java, main/resources, and main/webapp.
- The main/java directory stores Java source files, while main/webapp stores JSP pages and Web resources.
- External library JAR files required by applications are stored inside the WEB-INF/lib directory.
- WAR files are created using the JDK jar utility and can also be manually extracted using command-line switches.
- Web applications can be deployed either through the Tomcat console or by copying WAR files into the webapps directory.
- JSP pages can be created in Eclipse using the JSP wizard and executed directly on the configured Tomcat server.
- Eclipse automatically compiles applications and supports running projects directly inside internal or external Web browsers.

Session 4 – Jakarta Servlets

- Jakarta Servlets are server-side Java classes used for processing client requests and generating dynamic Web responses.
- Servlets operate inside servlet containers, which manage execution, request handling, security, and servlet lifecycle operations.
- The Servlet API provides classes and interfaces required for communication between clients, servers, and Web applications.
- Servlet architecture involves clients sending requests to servers, which forward them to servlets through the Web container.
- The servlet lifecycle begins with initialization through the init() method called only once during servlet creation.
- Client requests are processed using the service() method, which delegates processing to doGet() or doPost() methods.
- The destroy() method is called once before servlet removal to release resources and clean up processing tasks.
- HttpServletRequest objects store client request information such as parameters, headers, cookies, and session details.
- HttpServletResponse objects are used to generate responses, send output, set content types, and redirect clients.
- getWriter() is used for sending textual output, while getOutputStream() is used for binary response data.

- `sendRedirect()` redirects requests to another resource or page by sending a new response to the client browser.
- `RequestDispatcher` enables communication between resources by forwarding requests or including responses from other servlets or JSP pages.
- Servlet initialization parameters are configured using deployment descriptors or annotations and accessed during servlet execution.
- Session tracking is necessary because HTTP is stateless and cannot remember user information between requests automatically.
- Hidden form fields store session-related information within forms and transfer values between pages during form submission.
- Cookies are small pieces of data stored in browsers and used for session tracking and user identification.
- The Cookie API provides methods for creating, storing, retrieving, and managing cookies within servlet applications.
- `HttpSession` objects allow applications to store user-specific data across multiple requests during active sessions.
- Sessions can be invalidated manually to remove stored data and terminate user interaction securely.
- JSON Binding and serialization convert Java objects into JSON format for data exchange in modern Web applications.

Session 5 – Jakarta Server Pages (JSP)

- Jakarta Server Pages (JSP) simplify dynamic Web development by combining HTML markup with Java server-side processing capabilities.
- JSP was formerly known as Java Server Pages before transitioning into Jakarta Server Pages under Jakarta EE.
- JSP pages run inside servlet containers such as Apache Tomcat and are internally converted into servlets during execution.
- JSP separates presentation logic from business logic, reducing tight coupling commonly seen in servlet-only applications.
- JSP is preferred for presentation and UI generation, while servlets are preferred for processing and application control logic.
- JSP lifecycle includes translation into servlets, compilation, initialization, request processing, and destruction by the container.
- JSP architecture involves browsers sending requests to servers, which forward processing tasks to JSP containers and servlets.
- JSP scripting elements allow embedding Java code directly into Web pages for generating dynamic content.
- JSP directives provide configuration instructions to containers regarding page settings, imports, buffering, and included resources.
- JSP standard actions perform predefined operations such as including files, forwarding requests, and managing JavaBeans.
- The `<jsp:include>` action dynamically inserts content from another resource into the current JSP page during execution.
- The `<jsp:forward>` action forwards requests from one JSP page or servlet to another resource for processing.
- JSP implicit objects are automatically available objects provided by the container without explicit declaration inside JSP pages.
- Common JSP implicit objects include `request`, `response`, `session`, `application`, `out`, `config`, and `pageContext`.
- The `request` object stores client request information, while the `response` object manages server responses sent to clients.
- The `session` object stores user data across multiple requests until the session expires or becomes invalidated.

- The application object shares data globally between all users and pages within the Web application.
- The out object represents the response output stream used for displaying generated content on browser pages.
- JSP Expression Language (EL) simplifies accessing data, objects, operators, and expressions without writing complex Java code.
- EL expressions use `${}` syntax and support accessing request headers, JavaBeans properties, application data, and operators.
- JSP EL improves readability, reduces scripting complexity, and allows cleaner interaction with dynamic application content.

Session 6 – Expression Language (EL) and JSTL

- JSP Expression Language (EL) simplifies accessing variables, objects, request data, and application attributes without Java scripting code.
- EL expressions use `${}` syntax to retrieve values from scopes such as page, request, session, and application.
- EL automatically searches scopes in order and returns matching attribute values without explicit object declarations.
- EL supports arithmetic, relational, logical, conditional, and empty operators for processing dynamic application data.
- Arithmetic operators in EL include addition, subtraction, multiplication, division, and modulus for numeric calculations.
- Relational operators compare values using operators such as greater than, less than, equal to, and not equal.
- Logical operators such as and, or, and not are used for evaluating conditional expressions in JSP pages.
- EL implicit objects provide access to parameters, headers, cookies, initialization parameters, and context information automatically.
- The param object retrieves request parameters, while header retrieves HTTP header values from client requests.
- EL improves readability and maintainability by reducing the need for embedded Java code inside JSP pages.
- JSTL stands for Jakarta Standard Tag Library and provides reusable tags for common JSP operations and logic.
- JSTL removes the need for excessive Java scripting by replacing loops, conditions, and formatting operations with XML-like tags.
- JSTL Core tags handle variable creation, conditional statements, URL management, and iteration processing within JSP pages.
- The `<c:if>` tag executes content conditionally, while `<c:choose>` works similarly to switch-case structures in Java.
- The `<c:forEach>` tag iterates through collections, arrays, or ranges for displaying repetitive dynamic content.
- JSTL Formatting tags support localization, date formatting, number formatting, and internationalization features in Web applications.
- JSTL SQL tags provide database interaction support, although modern enterprise applications prefer JDBC or JPA solutions.
- JSTL XML tags process XML documents, parse XML data, and support XPath expressions inside JSP applications.
- JSTL Functions tags provide reusable utility functions such as string length, substring operations, and string replacement.
- EL and JSTL together promote cleaner, reusable, and more maintainable JSP-based enterprise Web applications.

Session 7 – JDBC (Java Database Connectivity)

- JDBC is a Java API used for connecting Java applications with relational database management systems.
- JDBC allows applications to execute SQL queries, retrieve data, update records, and interact with databases dynamically.
- JDBC architecture includes applications, JDBC API, JDBC DriverManager, database drivers, and database servers.
- JDBC drivers translate Java database requests into database-specific communication protocols understood by database systems.
- Type 1 JDBC drivers use ODBC bridges, while Type 4 drivers communicate directly with databases using native protocols.
- Type 4 JDBC drivers are most commonly used because they provide better portability, performance, and platform independence.
- JDBC applications typically follow steps including driver loading, connection creation, statement execution, and connection closing.
- The DriverManager class manages database drivers and establishes communication between applications and databases.
- The Connection interface represents an active database connection used for executing SQL commands and transactions.
- The Statement interface executes static SQL queries directly against the connected relational database system.
- The PreparedStatement interface supports precompiled parameterized SQL queries for better security and performance.
- Prepared statements help prevent SQL Injection attacks by separating SQL logic from user-supplied input values.
- The ResultSet object stores query results and provides methods for navigating rows and retrieving column values.
- JDBC supports transaction management through commit and rollback operations to ensure database consistency and reliability.
- Auto-commit mode automatically saves changes after every SQL operation unless explicitly disabled by developers.
- Batch processing allows multiple SQL commands to execute together for improved efficiency and reduced database overhead.
- Metadata classes provide information about database structures, tables, columns, and query results during runtime execution.
- Exception handling in JDBC uses SQLException objects containing error codes, messages, and database-specific information.
- Connection pooling improves performance by reusing existing database connections instead of creating new ones repeatedly.
- JDBC remains a fundamental technology for enterprise database communication within Java and Jakarta EE applications.

Session 8 – Java Persistence API (JPA)

- JPA is a Jakarta EE specification used for managing relational data using Java objects instead of direct SQL operations.
- JPA simplifies Object Relational Mapping (ORM) by mapping Java classes and objects to relational database tables.
- ORM reduces impedance mismatch between object-oriented programming concepts and relational database storage structures.
- Entity classes represent database tables, while object instances represent individual rows stored within those tables.
- The @Entity annotation marks a Java class as a persistent entity managed by the JPA provider.

- The @Table annotation specifies the corresponding database table name associated with an entity class.
- Primary keys uniquely identify entity records and are declared using the @Id annotation.
- The @GeneratedValue annotation automatically generates primary key values using configured generation strategies.
- Entity relationships are represented using annotations such as @OneToOne, @OneToMany, and @ManyToMany.
- JPA providers such as Hibernate implement JPA specifications and handle communication between applications and databases.
- The EntityManager interface performs persistence operations such as insert, update, delete, and query execution.
- Persistence units define database configurations, entity classes, and provider information within persistence.xml.
- JPQL (Java Persistence Query Language) queries entities and objects rather than querying database tables directly.
- Named queries are predefined reusable JPQL queries stored inside entity classes using annotations.
- Entity lifecycle states include new, managed, detached, and removed depending on persistence context management.
- Transactions ensure consistency when performing multiple database operations and are managed through transaction APIs.
- Lazy loading delays retrieving related entity data until explicitly requested, improving performance and resource management.
- Eager loading retrieves related entity data immediately when the parent entity is loaded into memory.
- Cascading operations automatically propagate persistence actions such as persist, merge, and remove to related entities.
- JPA significantly reduces boilerplate JDBC code and improves maintainability of enterprise database-driven applications.

Session 9 – Enterprise JavaBeans (EJB)

- Enterprise JavaBeans (EJB) are server-side components used for implementing scalable and transactional business logic.
- EJB simplifies enterprise application development by providing built-in transaction management, security, and lifecycle handling.
- EJB components run inside EJB containers that manage concurrency, transactions, security, and resource allocation automatically.
- Session Beans represent temporary business processes and are classified into Stateless, Stateful, and Singleton session beans.
- Stateless Session Beans do not maintain client-specific state between requests and are highly scalable and reusable.
- Stateful Session Beans maintain client-specific conversational data across multiple method calls during active sessions.
- Singleton Session Beans maintain a single shared instance across the entire application lifecycle and all connected clients.
- Message-Driven Beans process asynchronous messages received from messaging systems such as JMS queues or topics.
- The @Stateless, @Stateful, and @Singleton annotations define different session bean types within enterprise applications.
- Dependency Injection automatically injects enterprise resources and EJB references into managed application components.
- Container-managed transactions automatically handle transaction boundaries without requiring manual transaction management code.
- Bean-managed transactions allow developers to explicitly control commit and rollback operations programmatically.

- Interceptors allow developers to execute additional logic before or after business method execution automatically.
- EJB security supports role-based access control and authentication for protecting enterprise business services.
- Timers and scheduling services allow EJB components to execute tasks automatically at configured intervals or times.
- Remote interfaces enable EJB components to communicate with distributed systems running on different servers or machines.
- Local interfaces optimize communication between components running inside the same application server environment.
- EJB lifecycle methods manage initialization, activation, passivation, and destruction of enterprise bean instances.
- Jakarta EE applications commonly integrate EJB with JPA, Servlets, REST APIs, and messaging services.
- EJB improves enterprise reliability, scalability, maintainability, and modularity in large distributed application systems.

Session 10 – Java Message Service (JMS)

- Java Message Service (JMS) is a messaging API used for asynchronous communication between distributed application components.
- JMS allows applications to exchange messages reliably without requiring direct simultaneous communication between systems.
- Messaging systems improve scalability, fault tolerance, and loose coupling between enterprise application components.
- JMS architecture includes producers, consumers, destinations, brokers, and message-oriented middleware systems.
- Message producers create and send messages, while message consumers receive and process incoming messages asynchronously.
- JMS supports two communication models known as Point-to-Point and Publish-Subscribe messaging architectures.
- Point-to-Point messaging uses queues where one consumer receives and processes each message sent by producers.
- Publish-Subscribe messaging uses topics where multiple subscribers receive copies of published messages simultaneously.
- JMS messages contain headers, properties, and message bodies storing application-specific information and metadata.
- Common JMS message types include TextMessage, ObjectMessage, BytesMessage, MapMessage, and StreamMessage.
- Message brokers manage message routing, delivery, storage, persistence, and communication reliability between applications.
- Persistent messages remain stored until delivered successfully, improving reliability during server or network failures.
- Non-persistent messages improve performance but may be lost if messaging systems fail unexpectedly during communication.
- JMS sessions manage message production, message consumption, transactions, and acknowledgment processing between clients and brokers.
- Message acknowledgment confirms successful message delivery and processing between messaging systems and consumers.
- JMS supports asynchronous processing, allowing applications to continue execution without waiting for immediate responses.
- Message-driven beans integrate EJB with JMS for enterprise-level asynchronous message processing within Jakarta EE applications.
- JMS transactions ensure message delivery consistency and support rollback operations during processing failures.

- Enterprise systems commonly use JMS for notifications, payment processing, distributed systems, and event-driven architectures.
- JMS improves reliability, scalability, and flexibility when integrating independent enterprise applications and services.

Session 11 – Web Services

- Web services enable communication and data exchange between different applications running on different platforms and technologies.
- Web services support interoperability by allowing systems written in different programming languages to communicate effectively.
- Two major types of Web services are SOAP-based Web services and RESTful Web services architectures.
- SOAP stands for Simple Object Access Protocol and uses XML for structured message communication between applications.
- SOAP Web services are platform-independent and support high security, reliability, and standardized communication protocols.
- SOAP architecture includes SOAP Envelope, Header, Body, and Fault elements for organizing request and response messages.
- WSDL stands for Web Services Description Language and describes Web service operations, methods, and communication details.
- UDDI is a registry system used for publishing and discovering available Web services within distributed environments.
- REST stands for Representational State Transfer and uses standard HTTP methods for resource-based communication.
- RESTful services are lightweight, scalable, and easier to develop compared to SOAP-based Web service architectures.
- REST commonly uses JSON for data exchange because JSON is smaller, faster, and easier to process than XML.
- REST APIs use HTTP methods such as GET, POST, PUT, DELETE, and PATCH for performing operations on resources.
- REST resources are identified using Uniform Resource Identifiers (URIs) that uniquely locate application resources.
- Stateless communication in REST ensures each request contains all information required for processing independently.
- Jakarta EE supports RESTful Web services using Jakarta RESTful Web Services APIs and annotations.
- The `@Path` annotation defines resource paths, while `@GET` and `@POST` define supported HTTP methods.
- Web services are commonly used in enterprise integration, mobile applications, cloud systems, and distributed architectures.
- Advantages of Web services include interoperability, reusability, scalability, and easier integration across heterogeneous systems.
- SOAP services provide stronger standards and security, while REST services provide simplicity and improved performance.
- Modern enterprise applications widely prefer RESTful services due to lightweight communication and easier client integration.

Session 12 – Security in Web Applications

- Web application security protects applications, servers, users, and data from unauthorized access and cyber threats.
- Authentication verifies user identity, while authorization determines what authenticated users are allowed to access or perform.

- Common authentication methods include username-password systems, tokens, biometrics, and multi-factor authentication mechanisms.
- Session management securely maintains user identity and activity during interactions with enterprise Web applications.
- HTTPS uses SSL/TLS encryption to secure communication between browsers and servers against interception or tampering attacks.
- Cookies are often secured using flags such as Secure and HttpOnly to prevent unauthorized access and script exploitation.
- SQL Injection attacks occur when attackers insert malicious SQL code into user input fields and database queries.
- Prepared statements and parameterized queries help prevent SQL Injection by separating SQL commands from input values.
- Cross-Site Scripting (XSS) attacks inject malicious scripts into Web pages viewed by other users through browsers.
- Input validation and output encoding help prevent malicious scripts from executing within application interfaces or browsers.
- Cross-Site Request Forgery (CSRF) attacks trick authenticated users into performing unintended actions within applications.
- CSRF tokens are unique identifiers used to verify legitimate requests and prevent unauthorized request execution.
- Access control mechanisms restrict sensitive resources based on user roles, permissions, and authentication status.
- Passwords should be hashed and salted before storage to improve security and prevent unauthorized password recovery.
- Firewalls monitor and filter incoming and outgoing network traffic to protect enterprise systems against attacks.
- Security logging records suspicious activities, authentication attempts, and system events for monitoring and auditing purposes.
- Jakarta EE provides built-in security features including role-based authorization, authentication, and secure resource access.
- Enterprise applications should follow secure coding practices to reduce vulnerabilities and improve application reliability.
- Regular updates, security testing, and patch management are important for maintaining secure enterprise application environments.
- Web application security is essential for protecting sensitive user information, financial transactions, and enterprise systems.

Session 13 – WebSocket and Real-Time Communication

- WebSocket is a communication protocol enabling full-duplex real-time communication between clients and servers over TCP connections.
- Unlike HTTP, WebSocket maintains persistent connections allowing continuous bidirectional communication without repeated requests.
- WebSocket improves efficiency by reducing latency and overhead caused by repeated HTTP request-response communication cycles.
- Real-time applications such as chat systems, multiplayer games, and live notifications commonly use WebSocket technology.
- WebSocket communication begins with an HTTP handshake request that upgrades the connection to the WebSocket protocol.
- Once established, WebSocket connections remain open until explicitly closed by clients or servers during communication.
- WebSocket messages can be transmitted as text data or binary data depending on application communication requirements.
- Jakarta EE provides WebSocket APIs supporting annotated endpoints and programmatic endpoint configuration for applications.

- The `@ServerEndpoint` annotation defines WebSocket server endpoints accessible by connected client applications or browsers.
- Lifecycle annotations such as `@OnOpen`, `@OnMessage`, `@OnClose`, and `@OnError` handle communication events automatically.
- `@OnOpen` executes when a client establishes a connection, while `@OnMessage` processes received messages dynamically.
- WebSocket sessions manage connected clients and support communication between servers and active application users.
- Broadcasting allows servers to send messages simultaneously to multiple connected clients within enterprise applications.
- WebSocket reduces unnecessary network traffic compared to techniques such as repeated polling or long polling mechanisms.
- Security mechanisms such as authentication and encryption are important for protecting WebSocket communication channels.
- WebSocket supports event-driven architectures requiring immediate updates and real-time synchronization between connected systems.
- Enterprise systems use WebSocket for stock market updates, collaborative tools, live dashboards, and online gaming systems.
- Real-time communication improves responsiveness, interactivity, and user experience within modern Web and cloud applications.
- WebSocket integrates effectively with Jakarta EE applications, servlets, REST APIs, and enterprise messaging systems.
- Persistent communication through WebSocket enables scalable and efficient development of modern interactive enterprise applications.

Session 14 – Dependency Injection and CDI

- Dependency Injection is a design pattern where objects receive dependencies automatically instead of creating them manually.
- CDI stands for Contexts and Dependency Injection and is a Jakarta EE specification supporting managed object creation.
- CDI promotes loosely coupled applications by separating object creation from business logic and application functionality.
- Managed beans are CDI-managed objects whose lifecycle and dependencies are controlled by the Jakarta EE container.
- The `@Inject` annotation automatically injects dependencies into classes, reducing boilerplate object creation code.
- CDI scopes define bean lifecycles and determine how long managed objects remain available within applications.
- Request scope creates beans for a single HTTP request and destroys them after request processing completes.
- Session scope maintains bean data across multiple requests during active user sessions within enterprise applications.
- Application scope creates a single shared bean instance available throughout the entire application lifecycle.
- Dependent scope creates beans whose lifecycle depends directly on the lifecycle of the parent injected object.
- Qualifiers distinguish between multiple implementations of the same bean type during dependency injection operations.
- Producers generate objects dynamically using methods annotated with `@Produces` for advanced dependency injection scenarios.
- Interceptors execute additional logic such as logging, validation, or transactions before and after method execution.
- Events allow CDI components to communicate asynchronously without creating tight coupling between enterprise application modules.

- Alternatives provide multiple bean implementations and allow selecting specific implementations during deployment configuration.
- CDI integrates seamlessly with EJB, JPA, Servlets, REST APIs, and other Jakarta EE enterprise technologies.
- Dependency Injection improves maintainability, flexibility, testing, and scalability of enterprise software applications.
- CDI reduces code complexity by delegating lifecycle management and dependency resolution responsibilities to containers.
- Loose coupling achieved through CDI simplifies application modification, component replacement, and long-term enterprise maintenance.
- CDI is a core feature in modern Jakarta EE development and cloud-native enterprise application architectures.

Session 15 – Jakarta Faces (JSF)

- Jakarta Faces, previously called JavaServer Faces, is a component-based framework for building enterprise Web user interfaces.
- JSF simplifies user interface development using reusable UI components, server-side state management, and event-driven processing.
- JSF applications follow the Model-View-Controller architecture separating presentation, logic, and data management responsibilities.
- Facelets is the default JSF view technology used for creating reusable and maintainable XHTML-based Web pages.
- JSF pages use XML-style component tags for generating dynamic user interfaces and handling client interactions.
- Managed beans store application data and business logic used by JSF pages during request processing operations.
- The `@Named` annotation makes managed beans accessible from JSF pages using Expression Language expressions.
- JSF lifecycle includes restoring views, applying request values, validation, updating models, invoking actions, and rendering responses.
- Validation ensures user input meets application requirements before updating models or executing business logic operations.
- JSF converters transform data between user interface representations and Java object data types during processing.
- Navigation rules determine page transitions and workflow between different JSF application views and user interactions.
- Expression Language integrates with JSF for accessing bean properties, invoking methods, and displaying dynamic application data.
- Event handling in JSF processes user actions such as button clicks, form submissions, and component interactions.
- AJAX support in JSF enables partial page updates without reloading entire pages, improving responsiveness and performance.
- JSF UI components include forms, tables, buttons, input fields, messages, and navigation controls for enterprise interfaces.
- Facelets templates support reusable layouts and consistent page structures across multiple application pages and modules.
- JSF integrates effectively with CDI, EJB, JPA, and RESTful services within Jakarta EE enterprise applications.
- Built-in validation, conversion, and event processing reduce development complexity and improve application reliability.
- JSF supports rapid enterprise interface development through reusable components and container-managed processing capabilities.
- Jakarta Faces remains widely used for enterprise Web applications requiring server-side UI component management and integration.